SOFTWARE ENGINEERING MINI PROJECT • IEEE 829 STANDARD

Bank Management System

Master Test Plan & Verification Specification • Version 1.1

Document ID	`BMS-STP-001-v1.1`	**Release Status**	**Final / Approved**
Author & QA Lead	P Sachin (`Sachin-169`)	**Date of Issue**	November 10, 2026
Engineering Team	Niya Ann Joseph, Nimalan, Nimit, P Sachin	**Repository**	`Bank-Management-System-SE`

DOCUMENT REVISION HISTORY

Version	Date	Author	Summary of Changes
1.0	15-09-2026	P Sachin	Initial baseline test plan draft for Sprint 1 core banking features.
1.1	10-11-2026	P Sachin	Test cases added, BMS-NF-001 aligned to 2.0s threshold, schedule and approvals completed, lockout parameter verified, test runner details updated.

1. INTRODUCTION & PURPOSE

This Master Test Plan defines the formal verification and validation framework for the Bank Management System (BMS), a high-integrity, console-based banking application developed in C++17. The primary objective is to verify that all functional workflows, security invariants, atomicity constraints, and performance baselines satisfy the system specifications.

The scope of verification encompasses account lifecycle management, secure customer authentication, atomic debit/credit operations, transaction logging, administrator controls (customer search and deactivation), and persistent data restoration across unexpected process termination.

2. REFERENCE DOCUMENTS & STANDARDS

- **IEEE 829-2008:** Standard for Software and System Test Documentation.
- **Software Requirements Specification (SRS v1.0):** Approved functional and non-functional requirements catalog.
- **Software Architecture Document (SAD v1.0):** Modular structural specification and separation of concerns.
- **Test Case Specification (Test_Case_Specification_BMS.docx):** Full repository of 34 individual verification procedures.

3. REQUIREMENTS TO BE TESTED

The verification activities validate compliance against both functional and non-functional requirement sets:

Requirement ID	Description & Verification Objective
`BMS-F-001 – 004`	Authentication, credential rejection, 5-minute lockout enforcement, and dedicated administrator login.
`BMS-F-005 – 008`	Account creation with unique IDs, mandatory field validation, account view, and customer profile detail update.
`BMS-F-009 – 015`	Deposit validation, withdrawal balance checks, zero overdraft enforcement, and individual transaction history display.

Requirement ID	Description & Verification Objective
`BMS-F-016 – 020`	Atomic funds transfer between active accounts, dual ledger updates, and immutable audit record generation.
`BMS-F-021 – 024`	Admin customer search, account deactivation, data persistence across exit, and crash recovery handling.
`BMS-NF-001`	**Result of a transaction displayed within 2 seconds for at least 90% of operations.**
`BMS-NF-002 – 005`	Ledger consistency, input validation robustness, data persistence survival, and architectural modularity.
`BMS-SR-001 – 005`	Protected session enforcement, SHA-256 salted PIN storage, cross-account isolation, and transfer integrity on failure.

4. ITEMS EXCLUDED FROM TESTING

The following areas fall outside the operational scope of the current software sprint:

- Third-party payment gateway integration and credit card network settlement.
- Multi-branch inter-bank clearance network or distributed consensus protocols.
- Hardware Security Modules (HSM) deployment for physical key isolation.

5. TEST STRATEGY & APPROACH

Testing follows a multi-tier verification methodology spanning automated unit testing, component integration, simulated fault injection, and manual end-to-end regression. All automated builds are compiled with AddressSanitizer (`-fsanitize=address`) and UndefinedBehaviorSanitizer (`-fsanitize=undefined`) to guarantee zero memory corruption or undefined pointer arithmetic.

6. TEST TOOLS & ENVIRONMENT

> Unit tests are written as a lightweight assert-based runner (tests/test_*.cpp, run with `make test`) instead of GoogleTest or Catch2, so that the build needs only g++. Defects and execution status are tracked in Jira and exported to docs/defect_log.csv and docs/RTM_EXECUTION_STATUS.csv.

Host Environments: macOS (Darwin ARM64 Apple Silicon) and Linux (Ubuntu 22.04 LTS runner in GitHub Actions CI). **Toolchain:** GNU Compiler Collection (`g++`) with flags `-std=c++17 -Wall -Wextra -g` , Make build automation, and Git version control.

7. TEST SCHEDULE & MILESTONES

The test lifecycle followed the actual executed development timeline:

Milestone	Planned / Executed Dates
Test plan review and approval	15-09-2026
Test case design and review	by 12-10-2026
Environment setup	12-10 to 16-10-2026 (sprint dry run week)
Unit and integration test execution	19-10 to 23-10-2026 (Sprint 1) and 26-10 to 30-10-2026 (Sprint 2)
System test execution	after the final merge of Sprint 2
Performance and persistence testing	during Sprint 2 (`tests/test_perf.cpp`)

Milestone	Planned / Executed Dates
Regression cycle after defect fixes	after every merged fix
User acceptance testing / demonstration	date set by the instructor
Test summary report and sign-off	after the demonstration

8. HARDWARE & SOFTWARE SPECIFICATIONS

The software executes on standard POSIX-compliant environments requiring a minimum of 512 MB RAM, 50 MB persistent local storage, and terminal terminal emulation supporting ANSI escape sequences.

9. ROLES & RESPONSIBILITIES

Role	Assigned Team Member	Key Responsibilities
QA Lead	P Sachin (`Sachin-169`)	Test plan formulation, performance benchmarks, modularity review, defect management.
Development Lead	Niya Ann Joseph (`Pearl2Nyx`)	System architecture, transfer integrity validation, CI pipeline maintenance, PR merges.
Test Engineer	Nimalan (`nims-42`)	Transaction history implementation, deposit/transfer validation, deviation auditing.
Developer / Tester	Nimit (`LemonizDev`)	Administrator workflow implementation, account deactivation logic, authentication tests.

10. ENVIRONMENTAL & OPERATIONAL CONSTRAINTS

Data records reside in structured persistent binary/text formats (`data/accounts.dat` , `data/transactions.dat` , `data/audit.log`). Atomic file updates guarantee consistency against interrupted power or killed processes.

11. SECURITY & ACCOUNT LOCKOUT STRATEGY

> After 3 consecutive failed login attempts the account is blocked temporarily for 5 minutes (MAX_PIN_ATTEMPTS and LOCKOUT_SECONDS in include/common.h). The block is stored on disk, so restarting the application does not bypass it.

All account authentication mechanisms utilize cryptographic SHA-256 password hashing. Plaintext PINs are never stored in binary data files or recorded in debug logs.

12. SUSPENSION & RESUMPTION CRITERIA

Testing is suspended immediately if memory leaks or faults are flagged by AddressSanitizer, or if any existing unit test suite fails on the `main` branch. Execution resumes once a fix is peer-reviewed and passes GitHub Actions automated CI.

13. MINIMUM ACCEPTANCE TEST SUITE (CORE 10 CASES)

The full set of 34 test cases (one per requirement) is in the Test Case Specification (Test_Case_Specification_BMS.docx). The following ten cases cover functional, non-functional and security requirements and are the minimum set for acceptance:

TC ID	Requirement	Objective	Expected Result
TC-AUTH-01	`BMS-F-001`	Login with valid credentials	Account menu is shown
TC-AUTH-03	`BMS-F-003`	Three wrong PINs	Login blocked temporarily, also after a restart
TC-ACC-02	`BMS-F-006`	Empty name, invalid contact, short PIN	Each rejected; no account created
TC-DEP-02	`BMS-F-010`	Deposit 500 on balance 1000	Balance 1500
TC-WD-02	`BMS-F-013`	Withdraw more than the balance	Rejected; balance unchanged
TC-TR-03	`BMS-F-018`	Transfer 300 between two active accounts	Source 700, destination 800; total unchanged
TC-DATA-02	`BMS-F-024`	Start with a corrupted data file	Error message, no crash
TC-PERF-01	`BMS-NF-001`	20 timed runs per operation	90th percentile below 2 seconds
TC-ROB-01	`BMS-NF-003`	Letters, empty and very long input	Error and re-prompt, no crash
TC-SEC-02	`BMS-SR-002`	Inspect screen and data files for the PIN	PIN not echoed; only a hash is stored

14. NON-FUNCTIONAL PASS/FAIL CRITERIA

- **Performance:** Transaction completion and console display response time must be under **2 seconds** for at least 90% of samples (benchmarked under 0.5 ms via `tests/test_perf.cpp`).
- **Memory Integrity:** Zero memory leaks, zero buffer overflows, and zero undefined behavior instances under ASan / UBSan.
- **Modularity & Maintainability:** Strict architectural layer isolation; direct I/O streams (`cin` / `cout`) are restricted to `src/ui.cpp`.

15. DOCUMENT APPROVALS & SIGN-OFF

Role	Name	Signature / Status	Date
QA Lead	P Sachin (`Sachin-169`)	Approved (Signed: P Sachin)	10-11-2026
Dev Lead / PO	Niya Ann Joseph (`Pearl2Nyx`)	Approved (Signed: Niya Ann Joseph)	10-11-2026
Test Engineer	Nimalan (`nims-42`)	Approved (Signed: Nimalan)	10-11-2026
Developer / Tester	Nimit (`LemonizDev`)	Approved (Signed: Nimit)	10-11-2026
Course Coordinator	Faculty Evaluation Panel	Approved (Evaluated)	10-11-2026